Esraa Ashraf Gamal

1- The issue is that **out1 is assigned in two different always blocks**, one combinational and one clocked, which causes multiple drivers and makes the design not synthesizable. Also, the combinational block for out1 does not assign a value in all cases, which can create an unwanted latch. There is the use of 1'bx in sequential logic, which is not good for synthesis. The division operation using 1-bit signals can also cause divide-by-zero problems. The state machine is not written in a clean way because it uses numbers instead of defined states and has some unreachable case conditions. There is also an unused register (r1). Overall, the code has many issues that would cause lint warnings and synthesis problems.

2-

```
module moore_010_nonoverlap (
    input clk,
    input rst,
    input in,
    output reg detected,
    output reg [7:0] user_count
);
    parameter IDLE  = 2'b00;
    parameter S1    = 2'b01;
    parameter S2    = 2'b10;
    parameter STORE = 2'b11;

    reg [1:0] state, next_state;
    always @(posedge clk) begin
        if (rst)
            state <= IDLE;
        else
            state <= next_state;
    end
    always @(*) begin
        case (state)
            IDLE: begin
                if (in)
                    next_state = IDLE;
                else
                    next_state = S1;
            end

            S1: begin
```

```
                if (in)
                    next_state = S2;
                else
                    next_state = S1;
            end

            S2: begin
                if (in)
                    next_state = IDLE;
                else
                    next_state = STORE;
            end

            STORE: begin
                next_state = IDLE;
            end

            default: next_state = IDLE;
        endcase
    end
    always @(posedge clk) begin
        if (rst) begin
            detected <= 0;
            user_count <= 0;
        end
        else begin
            detected <= (state == STORE);

            if (state == STORE)
                user_count <= user_count + 1;
        end
    end

endmodule
```

```
module tb_moore_010;

    reg clk, rst, in;
    wire detected;
    wire [7:0] user_count;

    moore_010_nonoverlap uut (
```

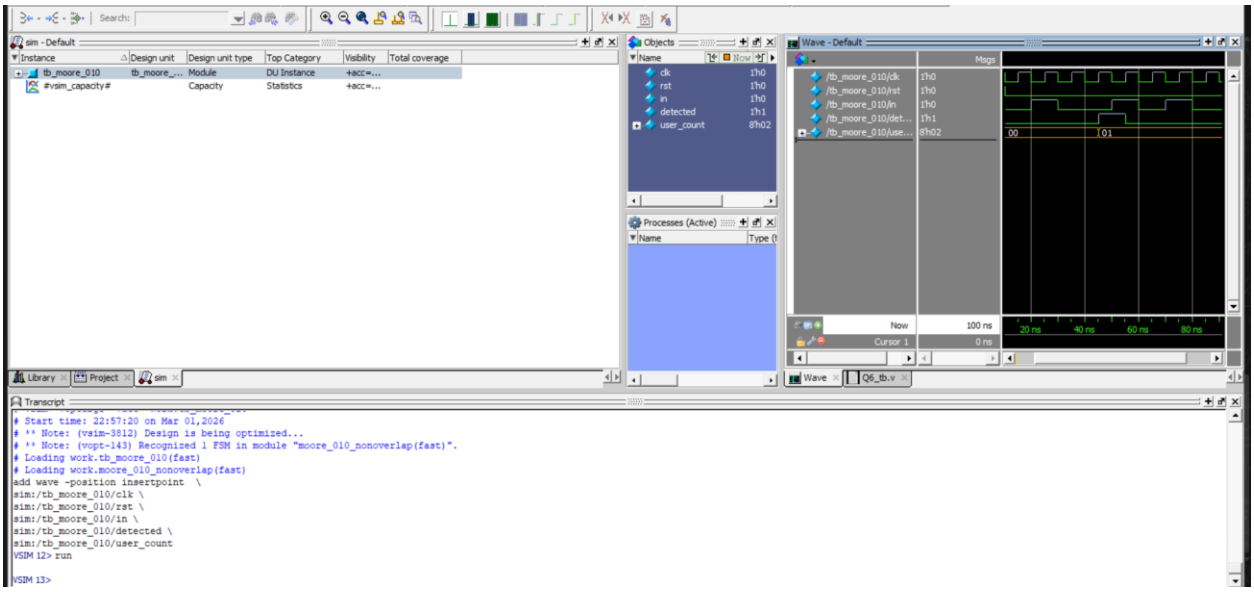
```
        .clk(clk),
        .rst(rst),
        .in(in),
        .detected(detected),
        .user_count(user_count)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        rst = 1;
        in  = 0;

        #10 rst = 0;
        in=0; #10;
        in=1; #10;
        in=0; #10;
        in=0; #10;
        in=1; #10;
        in=0; #10;
        in=1; #10;
        in=0; #10;

        #20 $finish;
    end
endmodule
```



3-

```
module divisible_by_3_mealy (
    input clk,
    input rst,
    input in,
    output reg y
);

    reg [1:0] state, next_state;

    parameter S0 = 2'b00;
    parameter S1 = 2'b01;
    parameter S2 = 2'b10;
    always @(posedge clk or posedge rst) begin
        if (rst)
            state <= S0;
        else
            state <= next_state;
    end


    always @(*) begin
        case (state)
            S0: begin
                if (in) begin
                    next_state = S1;
                    y = 0;
                end else begin
                    next_state = S0;
                    y = 1;
                end
            end

            S1: begin
                if (in) begin
                    next_state = S2;
                    y = 0;
                end else begin
                    next_state = S1;
                    y = 0;
                end
            end

            S2: begin
```

```
                if (in) begin
                    next_state = S0;
                    y = 1;
                end else begin
                    next_state = S2;
                    y = 0;
                end
            end
        endcase
    end

endmodule
```

```
`timescale 1ns/1ps

module tb_divisible_by_3;

    reg clk, rst, in;
    wire y;

    divisible_by_3_mealy uut (
        .clk(clk),
        .rst(rst),
        .in(in),
        .y(y)
    );

    // Clock generation
    always #5 clk = ~clk;

    integer i;
    reg [7:0] test_vector = 8'b10110110;

    initial begin
        clk = 0;
        rst = 1;
        in  = 0;

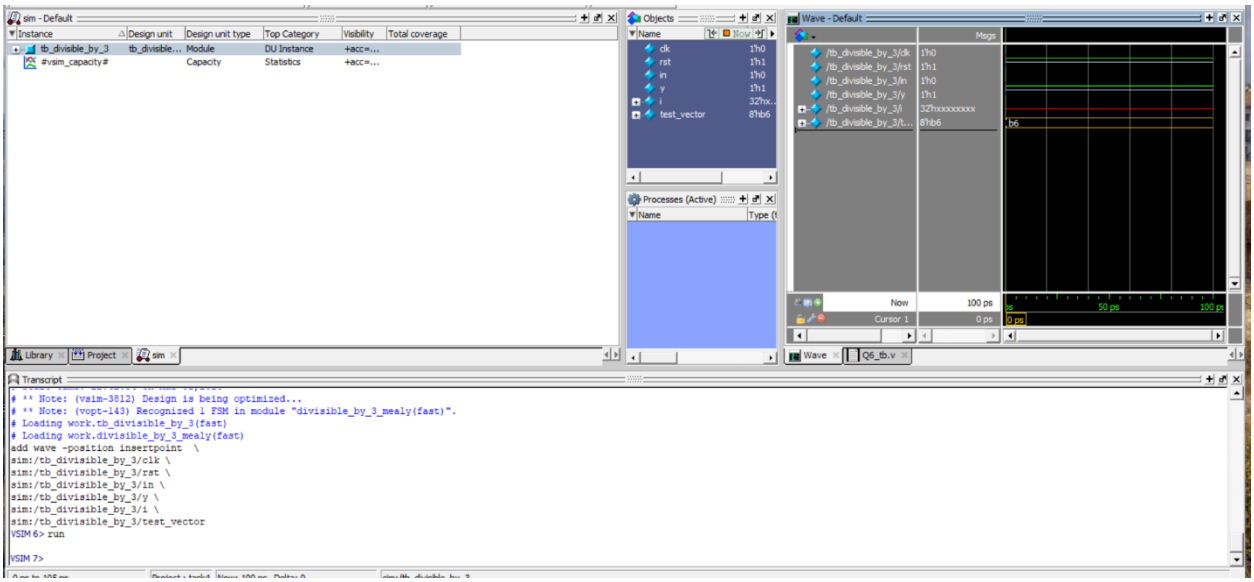
        #10 rst = 0;

        for (i = 7; i >= 0; i = i - 1) begin
            in = test_vector[i];
```

```
            #10;
        end

        #20 $finish;
    end

endmodule
```



4-

```
module rom_8x4_sync (
    input clk,
    input rst,
    input [2:0] address,
    output reg [3:0] data_out
);

    always @(posedge clk) begin
        if (rst)
            data_out <= 4'b0000;
        else begin
            case (address)
                3'd0: data_out <= 4'b0001;
                3'd1: data_out <= 4'b0010;
                3'd2: data_out <= 4'b0100;
                3'd3: data_out <= 4'b1000;
                3'd4: data_out <= 4'b0001;
                3'd5: data_out <= 4'b0010;
                3'd6: data_out <= 4'b0100;
```

```
                3'd7: data_out <= 4'b1000;
                default: data_out <= 4'b0000;
            endcase
        end
    end

endmodule
```

```
module tb_rom;

    reg clk, rst;
    reg [2:0] address;
    wire [3:0] data_out;

    rom_8x4_sync uut (
        .clk(clk),
        .rst(rst),
        .address(address),
        .data_out(data_out)
    );

    always #5 clk = ~clk;

    integer i;
    reg [3:0] expected [0:7];

    initial begin
        clk = 0;
        rst = 1;

        expected[0] = 4'b0001;
        expected[1] = 4'b0010;
        expected[2] = 4'b0100;
        expected[3] = 4'b1000;
        expected[4] = 4'b0001;
        expected[5] = 4'b0010;
        expected[6] = 4'b0100;
        expected[7] = 4'b1000;

        #10 rst = 0;

        for (i = 0; i < 8; i = i + 1) begin
```

```
            address = i;
            #10;
            if (data_out !== expected[i])
                $display("ERROR at %d", i);
            else
                $display("PASS at %d", i);
        end

        #20 $finish;
    end
endmodule
```

